两版拆分与「个人客户端优先」决策

> 日期：2026-10-11
> 触发：用户提出"分成两版（线上部署 / 个人客户端），并把主要精力先放在个人客户端本地使用"
> 前置：deliverables/研究报告-软件开发视角分析-2026-10-11.md（R→P→A→V 全链分析）

---

0. 先说现状：两版已经存在，但"local 版"并不是"个人客户端"

这是必须先澄清的事实——**不要从零拆两版。**

	现状 editions/local	**现状 editions/online**
监听	127.0.0.1:8720	0.0.0.0:8720（前接 nginx）
访问范围	本机一人	局域网多人
鉴权	无	nginx Basic Auth + 可选 QB_AUTH_TOKEN
部署	双击 启动题库.bat	Windows 宿主 + nginx（Docker 方式**导出不可用**）
数据	本机 outputs/bank.db	挂载卷
共享内核	bank/ pipeline/ webui/ frontend/ 零分叉，差异收敛在 webui/edition.py	同左

所以技术上的"两版"已经做好了，而且 editions/online/README.md 把三条硬约束写得非常诚实。

但用户要的"个人客户端"和现状 local 版不是一回事：

	现状 local 版	**用户要的"个人客户端"**
形态	本地 Web 服务 + 浏览器	桌面应用（双击即用）
前置	需装 Python、Node，需构建前端	**零安装**
导入	**全靠 4+ 条 CLI**	拖一个 DOCX 就行
数据	outputs/bank.db（藏在代码目录）	用户可见的"我的题库"文件

	现状 local 版	用户要的"个人客户端"
备份	手动复制	一键导出/恢复整库

结论：要做的是**把"技术上的两版"升级为"产品上的两版"**，而不是新建两套代码。

---

1. 重新定义两版（产品层）

维度	个人客户端版（优先）	线上部署版（后置）
目标用户	**单个老师**，管自己的库	办公室 2–十几位老师共享
使用形态	桌面应用，双击即用	浏览器访问内网地址
安装成本	**零安装**（自带运行时）	需一台常开 Windows 机 + nginx
数据归属	明确是"我的"	集中一处，多人共用
鉴权	不需要（单机）	共享口令（访问控制级）
并发导出	天然不存在（单人）	**必须串行**（`WinError 10048`）
首要矛盾	**采集/导入的门槛**	共享与权限
交付优先级	**P0 —— 全部精力**	P2 —— 明确推迟

---

2. 为什么"个人客户端优先"是正确的决策

这不只是偏好——**它直接消掉了刚做完的分析里一批悬而未决的问题**：

消掉的（原本是线上版的难题）

原问题	在个人客户端下
假设 A：谁建库、多久一次?	**答案自动明确**——自己建自己的库，不存在"管理员 vs 教师"的分工争议
无用户/角色体系	**不需要**（单机单用户）
并发导出 `WinError 10048`	**天然不存在**（一个人不会并发导出）
数据分发（`bank.db` 62MB 怎么给 10 个人）	**不需要**（各人自己的库）
轴 6「归属明确」/ P-09 贡献者署名	**价值大幅下降**——单用户下"这是谁建的题"意义不大 → 可降为 P2 或不做

原问题	在个人客户端下
审计日志、多人协同编辑	不需要

反而变得更关键的

原问题	在个人客户端下
P-05 Web 导入入口	**更关键**——没有管理员帮忙建库，**老师必须自己完成"导入旧卷"**。这是产品能否被用起来的第一道门
P-01 净时间实测	**更关键**——个人客户端没有"组织推动"的外力，**只有真的省时才会被持续使用**
数据备份/迁移	**新增刚需**——个人电脑会换、会坏，需要"一键导出整库"
离线可用	刚需（个人场景网络不可靠）

> **一句话**：线上版靠"组织推动"落地，个人客户端**只能靠"真的省时"**。所以 P-01 与 P-05 在个人客户端路线下**权重更高**，而归属/协作类需求权重更低。

---

3. 🟥 头号障碍：导出的外部依赖链（必须先解决）

这是"个人客户端"能不能成立的决定性问题。**实测确认**：

当前导出依赖链

```
bank/export.py
├─ 依赖 edsdk.py ──► 来自 workbuddy-builtin/skills/tencent-local-office-edit/edsdk.py
│   （WorkBuddy 插件的一部分，不属于本项目）
├─ 依赖 editor_sdk MCP 服务 ──► http://127.0.0.1:39099/mcp（需单独运行）
└─ 依赖 Word 或 WPS ──► COM 自动化
```

三个致命事实

1. `edsdk.py` **不在本项目里**——它来自 WorkBuddy 的内置插件。个人客户端**不能假设老师的电脑装了 WorkBuddy**。
2. `editor_sdk` **需要手动启动**——`webui/config.py` 的提示语是"**请先启动 editor_sdk 后再使用导出功能**"。老师不会去启动一个叫 `editor_sdk` 的服务。

3. **两个启动脚本都没有拉起它**——`editions/local/启动题库.bat` 与 `editions/online/start-server.bat` 里**都没有** `editor_sdk/edsdk` 相关代码（已实测 grep 确认）。

> **现状等于**：老师双击启动题库 → 一切正常 → 点"导出" → 报错"未检测到 editor_sdk 服务" → **不知道该怎么办**。
> 这是个人客户端的**头号落地障碍**，且**与 P-05（导入）同等重要**。

三条可行路径

路径	做法	优点	风险
A. 自动拉起	客户端启动时检测 39099 → 未运行则自动启动 → 退出时关闭	改动小，复用现有高质量导出链路	`editor_sdk` 是外部组件，**分发与授权存疑**；老师机器上可能根本没有
B. 自研生成器	用 `python-docx` + 手写 OOXML 直接生成 DOCX，**不依赖 editor_sdk**	完全自包含，可打包分发	**风险极高**——现有实现踩过的坑（`<a:graphicData>` 缺 `uri` 导致 Word 打不开、TAB 控制字符、HTML 实体残留、制表位）**都要重新踩一遍**；且 Word/WPS 仍是渲染必需的
C. 双路径降级	有 editor_sdk 走高质量路径；没有则用内置生成器（功能降级但可用）	兼顾	两套导出实现需**长期并行维护**，是最贵的方案

建议

先做 A（自动拉起）+ 明确的环境自检与降级提示，因为：

- 现有导出链路已被**双端实测验证过**（Word/WPS 改字符→保存→重开），重写风险远大于收益；
- A 的工程量最小，且**可以立刻让"双击即用"成立**；
- B/C 属于"如果要脱离 WorkBuddy 生态分发"时才需要，属后续决策。

但必须先确认一件事：`editor_sdk` 与 `edsdk.py` 的**分发许可**——能否随个人客户端一起交付给老师？这决定了 A 是否可行。**这是需要你去确认的业务/授权问题，不是技术问题。**

---

4. 优先级重排（个人客户端优先后）

优先级	需求	变化

优先级	需求	变化
P0	**P-00（新增）• 导出零配置** —— 客户端启动自动拉起依赖 + 环境自检 + 明确降级提示	**新增**（原分析未识别，因假设了"用户懂技术"）
P0	**P-01 • 端到端净时间实测**	保持（个人场景下权重**上升**）
P0	**P-05 • 零培训首次产出（DOCX 导入）**	保持（个人场景下**更关键**）
P0	**P-12（新增）• 零安装打包 + 一键启动**	**新增**（"个人客户端"的定义本身）
P1	素材标注覆盖率、元数据维度、难度检索、版式图示保真、答案覆盖补录	保持
P2 / 不做	**P-09 贡献者署名（原 P1）**	**降级**——单用户下价值低
推迟	用户/角色体系、并发导出队列、数据分发、审计日志、多人协同	**明确推迟**（线上版专属）

---

5. 技术路径（个人客户端）

环节	建议	说明
打包	PyInstaller 或内嵌 Python 运行时	目标：**老师机器上不需要装 Python/Node**
前端	构建产物随包分发（`webui/static/`）	已有 `npm run build` 产物托管机制
启动	一键启动器：启动本地服务 → 打开浏览器/内置窗口 → **自动拉起 editor_sdk**	复用 `editions/local/启动题库.bat` 思路，但需**补齐依赖启动**
数据位置	从代码目录迁到用户目录（如 `%USERPROFILE%\题库\bank.db`）	个人客户端的数据**不该藏在程序目录里**
备份	一键"导出整库"/"恢复整库"	个人电脑会换会坏
首次体验	首次启动提供"导入我的旧卷"引导	对应 P-05

---

6. 风险与"不做什么"

风险

1. **Office/WPS 依赖不可消除**——公式渲染最终仍靠 Word/WPS 的 COM。**老师机器上必须有 WPS 或 Word**（教师电脑基本都有，但不能假设 100%）。
2. `editor_sdk` **分发授权未确认**——这是 P-00 能否落地的前提。
3. **打包体积**——内嵌 Python + 依赖可能数百 MB。
4. **双版维护成本**——虽然共享内核，但两套部署形态的测试矩阵会翻倍。**建议线上版在个人客户端验证通过前保持"可用但不投入"**。

不做什么

- **不重写导出链路**（现有链路已双端实测验证，重写风险 > 收益）。
- **不为线上版做用户/角色体系**（推迟到有明确多人需求时）。
- **不做"个人客户端 + 云端同步"**（会重新引入账号、鉴权、冲突合并的全部复杂度）。

---

7. 建议的下一步

1. **确认** `editor_sdk` **/** `edsdk.py` **的分发许可**（业务问题，卡点）。
2. **做 P-00 + P-12 的最小验证**：在一台"干净"的 Windows 机器上（只装 WPS，不装 WorkBuddy），尝试打包并跑通"双击 → 导入 DOCX → 组卷 → 导出可编辑 DOCX"。

- 这一步会**同时验证**：打包可行性、依赖自举、以及**假设 A**（老师能否自己完成建库）。

1. **再决定 P-01（净时间实测）的对照组设计**——个人场景下，对照应该是"这位老师原本怎么组卷"，而不是抽象的"手工组卷"。

> **一句话**：两版的技术骨架已经有了，真正要做的是**把"个人客户端"从"本地 Web 服务"升级为"零安装桌面应用"**——而这条路上**最大的坑不是代码，是导出依赖链里那个老师永远不会去启动的** `editor_sdk`。